DO NOW — RECALL

Compiler stages, linkers, loaders and libraries

Name: _____ Class: _____ Date: _____

1. Which program translates assembly mnemonics?

2. What does a compiler typically produce?

3. What is a runtime interpreter?

4. Simplify ¬(A∨B) with De Morgan's law, then evaluate A=0,B=1.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Assembler.

2. Model answer

Target/object code.

3. Model answer

A system that executes/analyses source or intermediate operations at runtime.

4. Model answer

¬A∧¬B; with A=0,B=1 gives 1∧0=0.

Vocabulary — know these terms

lexer — Recognises source tokens.

parser — Checks grammatical structure.

semantic analysis — Checks program meaning and types.

linker — Resolves symbols across modules.

loader — Places program in memory.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Model answer

Explain the reasoning and justify the output.

IMPROVE: Explain one correction to your own reasoning.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

The program calls PrintReport() from another object file. Which stage resolves the reference?

The linker finds its definition in the module/library and fixes up the reference before execution.



WEDO · STEP 2

NEW LEARNING

Read, annotate and explain

Understand

Lexical analysis identifies tokens, while syntax analysis checks grammar and generates a parse structure.

Apply

Semantic analysis checks rules beyond grammar such as type compatibility or declared identifiers; optimisations can transform intermediate code.

Evaluate

A linker combines object files and resolves symbols/libraries; a loader places the executable into memory for execution.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



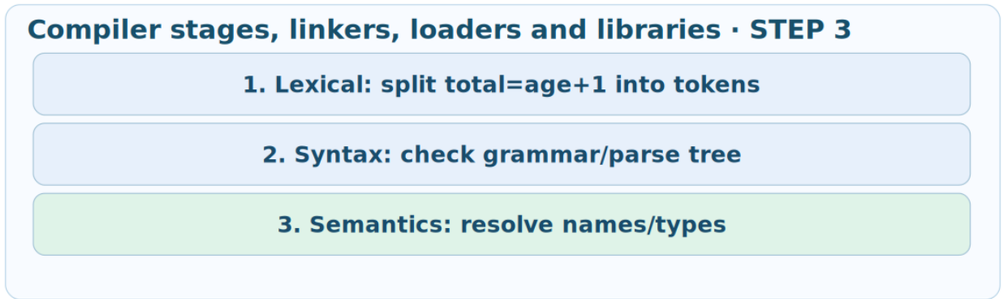
Compiler stages, linkers, loaders and libraries · STEP 1

New Learning



Compiler stages, linkers, loaders and libraries · STEP 2

New Learning



Compiler stages, linkers, loaders and libraries · STEP 3

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A parser guarantees that every variable in a syntactically valid program was declared.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

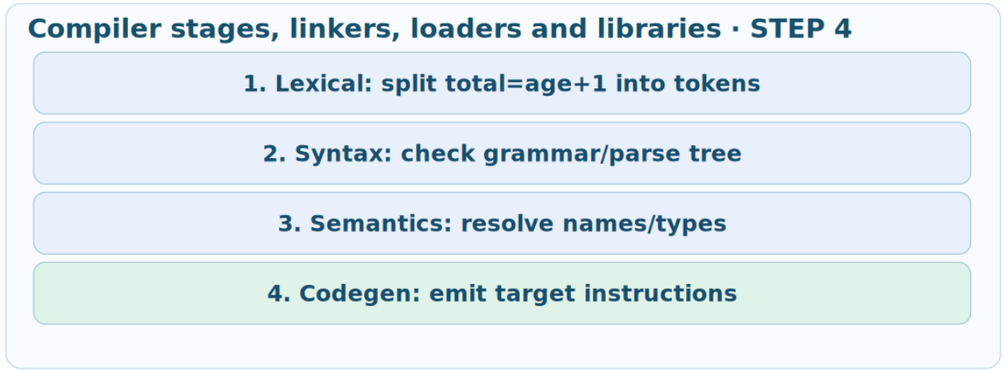
Question 3

A source file has valid syntax but calls undefined MethodX(). Explain which analyses or stages may detect failure, and why. [6]

New Learning · original lesson diagrams

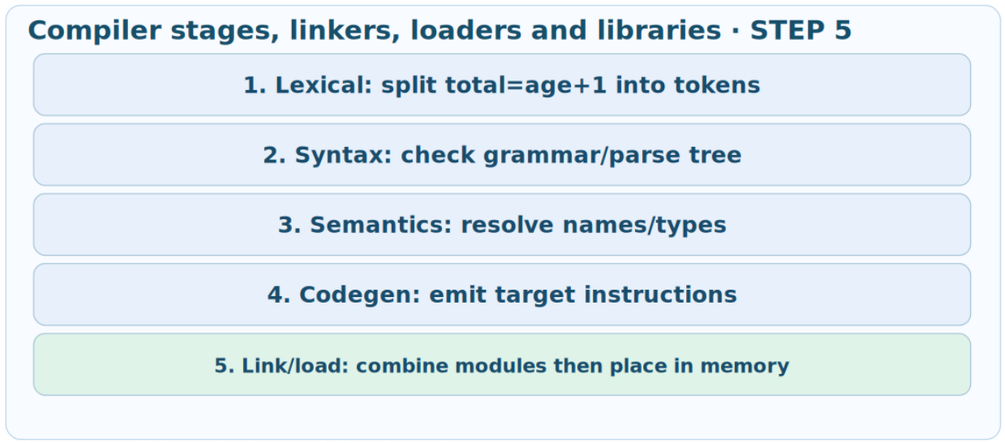
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Compiler stages, linkers, loaders and libraries · STEP 4

New Learning



Compiler stages, linkers, loaders and libraries · STEP 5

I DO — WORKED EXAMPLE

Follow the worked method

Analyse `total = count + 1;` into tokens and syntax structure.

Predict first

The lexer recognises tokens

Model answer

parser checks that they form a valid assignment statement.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Distinguish lexical, syntax and semantic analysis. [6]

Question 2

Describe the roles of code generation, linking and loading. [6]

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict first The linker finds its definition in the module/library and fixes up the reference before execution.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

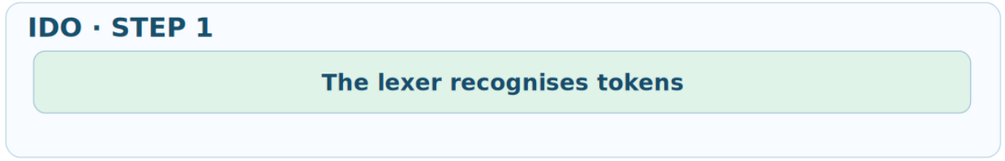
IMPROVE — Write one precise correction or stronger explanation in your own answer.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Analyse `total = count + 1;` into tokens and syntax structure.

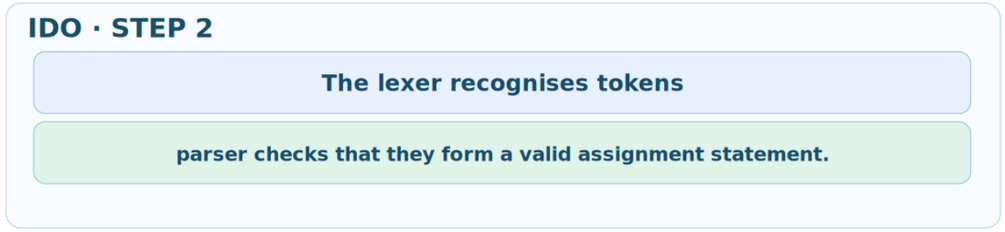
The lexer recognises tokens



IDO · STEP 1

Analyse `total = count + 1;` into tokens and syntax structure.

The lexer recognises tokens



IDO · STEP 2

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

The program calls PrintReport() from another object file. Which stage resolves the reference?

The linker finds its definition in the module/library and fixes up the reference before execution.



WEDO · STEP 1